SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1723**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 30%

Duration : 45 Minutes

QUESTIONS: 2 **Total Marks:20**

Annexure A

Descriptive Written Test

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Descriptive Written Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

1. Introduction

An intelligent agent is a system that perceives its environment through sensors and performs actions through actuators to achieve specific objectives. In a smart home automation system, the agent manages lighting, temperature and security based on user behaviour and environmental conditions.

The system receives real-time inputs such as motion detection, temperature changes and time of day. It can also learn user preferences over time to improve comfort, energy efficiency and security.

The four important types of intelligent agents are:

1. Simple Reflex Agent
2. Model-Based Reflex Agent
3. Goal-Based Agent
4. Utility-Based Agent

2. Types of Intelligent Agents and Their Functions

2.1 Simple Reflex Agent

A simple reflex agent selects actions based only on the current percept. It follows condition–action rules and does not consider past experiences or maintain an internal model of the environment.

Working principle:

The agent checks the current input and immediately performs the corresponding action.

Examples in a smart home:

- If motion is detected in a room, switch on the lights.
- If the temperature exceeds a predefined limit, switch on the cooling system.
- If an unauthorized person is detected, activate an alarm.

Advantages:

- Simple to design and implement.
- Responds quickly to real-time inputs.
- Requires less memory and computational power.

Disadvantages:

- Cannot remember previous situations.
- Cannot learn user preferences by itself.
- May make unsuitable decisions when the current input is insufficient.

Analysis: A simple reflex agent is useful for basic tasks that require immediate responses, such as motion-triggered lighting and alarm activation. However, it is not sufficient for managing the entire smart home intelligently.

2.2 Model-Based Reflex Agent

A model-based reflex agent maintains an internal state of the environment. It uses the current percept and stored information about previous observations to make decisions.

Working principle:

The agent updates its internal state whenever new information is received and selects an action based on that state.

Examples in a smart home:

- Remembers whether a room is occupied based on recent motion observations.
- Maintains the current temperature and air-conditioning status.
- Tracks whether doors and windows are open before activating security settings.

Advantages:

- Works better in partially observable environments.
- Uses stored information to make more informed decisions.
- Can consider changes in the environment over time.

Disadvantages:

- Requires memory to maintain its internal state.
- The internal model must be updated accurately.
- Does not necessarily compare different future goals or evaluate their overall benefits.

Analysis: A model-based agent is useful when smart home decisions depend on the current situation and previous observations. For example, it can avoid switching off the air conditioner immediately when the temperature sensor gives a temporary unusual reading.

2.3 Goal-Based Agent

A goal-based agent selects actions by considering a desired goal or future state. It evaluates possible actions and chooses those that help achieve the goal.

Working principle:

The agent identifies the goal, considers possible actions and selects a suitable sequence of actions to reach the desired state.

Examples in a smart home:

- Maintains a comfortable room temperature.
- Ensures the doors are locked when the house is set to Away mode.
- Turns off unnecessary lights when the goal is to reduce energy consumption.

Advantages:

- Makes decisions based on desired outcomes.
- Can compare alternative action sequences.
- Adapts its actions when the goal changes.

Disadvantages:

- May require more computation to evaluate possible actions.
- Achieving one goal does not automatically guarantee the best balance between comfort, energy use and security.
- Requires clearly defined goals.

Analysis: A goal-based agent is suitable for tasks such as preparing the house for sleep or securing the home before the occupants leave.

2.4 Utility-Based Agent

A utility-based agent evaluates possible outcomes using a utility function. It selects actions that maximize expected utility according to the system's objectives.

Unlike a basic goal-based agent, it can compare several desirable outcomes and choose the one that provides the best overall balance.

Working principle:

The agent evaluates the expected benefits and costs of possible actions and chooses the action with the highest utility.

Examples in a smart home:

- Balances room comfort against electricity consumption.
- Adjusts lighting based on occupancy and energy efficiency.
- Gives security greater priority when suspicious activity is detected.
- Chooses a temperature setting that provides comfort without unnecessary energy use.

Advantages:

- Balances multiple objectives.
- Supports decisions involving trade-offs.
- Can make more flexible decisions under uncertainty when expected utility is used.

Disadvantages:

- Designing a suitable utility function can be difficult.
- Different objectives may conflict with each other.
- Requires reliable sensor information and appropriate evaluation of costs and benefits.

Analysis: A utility-based agent is especially useful when comfort, energy efficiency and security must be considered together.

3. Comparison of the Four Intelligent Agents

Agent type	Main feature	Smart home application
Simple Reflex	Uses current input and rules	Motion-based lighting
Model-Based	Maintains an internal state	Tracks room occupancy and temperature
Goal-Based	Works toward a specific goal	Secures the house in Away mode
Utility-Based	Maximizes overall utility	Balances comfort, energy and security

4. Most Effective Agent or Hybrid Approach

A hybrid approach combining model-based and utility-based agents would be the most effective for the smart home automation system.

The model-based component maintains information about occupancy, temperature, door status and previous observations. The utility-based component evaluates possible actions and balances comfort, energy efficiency and security.

For example, when the occupants leave the house, the system can recognize the Away mode, switch off unnecessary lights, adjust the air conditioner and activate the security system. If suspicious activity is detected, security can receive a higher priority.

Simple reflex rules can also be included for urgent tasks, such as activating an alarm when a defined security condition is detected.

The hybrid approach is effective because it combines quick responses, environmental awareness and intelligent decision-making.

5. Conclusion

Intelligent agents play an important role in smart home automation. Simple reflex agents handle immediate responses, model-based agents maintain environmental information, goal-based agents work toward desired outcomes, and utility-based agents balance competing objectives.

Therefore, a hybrid system combining model-based and utility-based agents, supported by simple reflex rules for urgent events, is the most suitable solution for improving comfort, energy efficiency and security.

2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem. Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

   1. Introduction

   Uninformed search is a search technique that explores possible states without using a heuristic to estimate how close a state is to the goal.

In this problem, a robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. The robot must explore possible paths, identify obstacles and select a route to reach the exit.
Two important uninformed search strategies are:
1. Uniform Cost Search (UCS)
2. Iterative Deepening Search (IDS)

These algorithms differ in their exploration methods, memory requirements and ability to handle different movement costs.

2. Uniform Cost Search (UCS)

Definition

Uniform Cost Search is an uninformed search algorithm that expands the node with the lowest accumulated path cost first.
It uses a priority queue to store nodes according to their path costs.
The path cost is represented as:
\[g(n)=\sum_{i=1}^{n}c_i \]
Where:
- \(g(n)\) is the accumulated cost of reaching node \(n\).
- \(c_i\) is the cost of an individual movement.

Working Procedure

1. Place the starting position in the priority queue with cost zero.
2. Remove the node with the lowest accumulated cost.
3. Check whether the node is the exit.
4. If it is not the exit, generate its valid neighbouring cells.
5. Calculate the accumulated cost of reaching each neighbour.
6. Update the priority queue when a cheaper path to a state is found.
7. Repeat until the exit is reached or no reachable states remain.

Example

Consider a robot exploring a maze with several possible routes. One route may contain four movements, each costing 1, while another route contains three movements with costs 1, 1 and 5.
The total costs are:
Route A:
\[C_A=1+1+1+1=4 \]
Route B:
\[C_B=1+1+5=7 \]
Although Route B has fewer movements, UCS prefers Route A because its total cost is lower.

Advantages of UCS

- Finds the least-cost path when movement costs are non-negative.
- Is complete when each action has a positive minimum cost and a solution is reachable.
- Handles situations in which different paths have different costs.
- Does not require a heuristic function.

Disadvantages of UCS

- Can require substantial memory to store the frontier and explored states.
- May expand many nodes before reaching the exit.

- Can be inefficient when the maze is large.
- In an unknown maze, the robot must discover the layout and update its map before it can reliably plan using UCS.

3. Iterative Deepening Search (IDS)

Definition
Iterative Deepening Search combines the depth-first exploration style of Depth-First Search with the increasing depth limits of repeated searches.
It begins with a small depth limit and gradually increases the limit until the goal is found.

Working Procedure
1. Start with a depth limit of zero.
2. Search the maze up to the current depth limit.
3. If the exit is not found, increase the depth limit by one.
4. Restart the search from the starting position.
5. Repeat until the exit is found or the search limit is reached.

Example
Suppose the exit is located four steps away from the starting position.
- Depth limit 0: Only the starting position is explored.
- Depth limit 1: The starting position and its immediate neighbours are explored.
- Depth limit 2: The search explores paths up to two steps deep.
- Depth limit 3: The search explores paths up to three steps deep.
- Depth limit 4: The search can reach the exit.

IDS repeatedly searches at increasing depths until it finds the goal.

Advantages of IDS
- Requires less memory than UCS in many large search spaces.
- Is complete when the branching factor is finite and the goal is reachable at a finite depth.
- Finds a shallowest solution when all actions have equal cost.
- Does not require a heuristic function.

Disadvantages of IDS
- Repeats searches from the starting position.
- Re-expands nodes that were visited in earlier iterations.
- Is not guaranteed to find the least-cost path when movement costs differ.
- May be inefficient if the maze contains many deep branches or if the robot must physically explore the maze without a stored map.

4. Comparison of UCS and IDS
Let:
- \(b\) = branching factor, or average number of successors per node.
- \(d\) = depth of the shallowest goal.
- \(C^*\) = cost of an optimal solution.
- \(\varepsilon\) = minimum positive action cost.

Feature	Uniform Cost Search	Iterative Deepening Search
Search method	Expands the lowest-cost path	Repeats depth-limited searches
Main data structure	Priority queue	Stack or recursive depth-first search
Time complexity	Exponential in solution cost for general bounded branching and positive step costs	\(O(b^d)\)
Space complexity	Exponential; stores frontier and explored states	\(O(bd)\)
Completeness	Complete with positive minimum step costs	Complete with finite branching and a reachable finite-depth goal
Optimality	Finds a least-cost path with non-negative costs under standard conditions	Finds a shallowest path; optimal when all step costs are equal
Main advantage	Handles different path costs	Low memory requirement
Main disadvantage	High memory usage	Repeated exploration

For UCS, a common worst-case bound for the number of generated nodes is exponential in the optimal solution cost, approximately \(O\left(b^{1+\lfloor C^*/\varepsilon\rfloor}\right)\) under standard assumptions.

5. Relationship to Intelligent Agent Types

5.1 Simple Reflex Agent

A simple reflex agent selects actions based on the current percept.
In the maze, it might follow a rule such as moving forward when the next cell is clear.
Limitation: It does not maintain a complete internal map or remember previously visited locations. Therefore, it may repeatedly enter the same dead ends.

5.2 Model-Based Agent

A model-based agent maintains an internal state of the environment and updates it using new observations.
In the maze, it can:

- Record explored cells.
- Mark discovered obstacles.
- Remember dead ends and visited locations.
- Update its internal map when new paths are discovered.

This agent is more suitable for an unknown maze because it can use accumulated knowledge instead of relying only on the current percept.

5.3 Goal-Based Agent

A goal-based agent chooses actions that help it reach a specified goal.
In the maze, the goal is to reach the exit. The agent considers available routes and selects actions that move it toward the exit.
However, the robot cannot plan accurately through unexplored areas unless it discovers the layout or uses assumptions about unknown cells.

5.4 Utility-Based Agent

A utility-based agent evaluates alternative outcomes and selects the action with the highest utility.
For example, if the robot must balance travel distance, movement cost and safety, a utility-based approach can compare these factors.
However, the problem does not specify different movement costs or risk levels, so a utility function is not essential for the basic maze task.

6. How Agent Design Affects Decision-Making

The design of an intelligent agent directly affects how it explores and solves the maze.
A simple reflex agent reacts quickly but may repeatedly visit the same locations. A model-based agent stores information about the environment and avoids unnecessary repeated exploration. A goal-based agent focuses on reaching the exit, while a utility-based agent can compare alternative routes according to their overall benefits and costs.
Since the maze is initially unknown, the robot should maintain an internal map, observe neighbouring cells, record obstacles and update its knowledge after every movement. This enables it to make better decisions as exploration progresses.

7. Most Effective Combination and Justification

The most suitable combination is a model-based agent with online exploration and Uniform Cost Search, especially when movement costs may differ.
The reasons are:

1. Supports unknown environments: The model-based agent records newly discovered cells and obstacles.
2. Finds a least-cost path: UCS expands paths in increasing order of accumulated cost.
3. Reduces repeated exploration: The internal map helps the robot recognize previously visited cells and dead ends.
4. Supports replanning: If the robot discovers that a route is blocked, it can update its map and search again.
5. Provides reliable decision-making: The robot combines current observations with stored knowledge when selecting its next action.

If every movement has the same cost and memory is severely limited, IDS is a suitable alternative because it finds a shallowest solution while using less memory. However, in a real unknown maze, IDS alone does not maintain the persistent map required for efficient physical exploration.
Important limitation: Standard UCS assumes that the search can access a represented state space and its transition costs. For a genuinely unknown maze, the robot must combine search with sensing, mapping and replanning. It cannot guarantee the globally optimal route before discovering the relevant parts of the maze.

8. Conclusion

Uniform Cost Search and Iterative Deepening Search are useful uninformed search strategies for maze-solving problems. UCS is preferable when the robot must minimize total path cost, while IDS is preferable when all movements have equal cost and memory efficiency is important.

A model-based intelligent agent is particularly suitable for an unknown maze because it maintains an internal map and updates its knowledge as it explores. Therefore, combining a model-based agent with online exploration and UCS provides an effective solution when the objective is to reach the exit with minimum accumulated cost.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks

Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand